

中国邮递员问题

第三次小班课个人资料



目录

1 定义概述

2 预备知识

3 算法思想

4 算法实现

5 求解过程

6 性能分析

7 实际应用



一. 定义概述

1. 定义

中国学者于20世纪50年代提出的一种典型的组合优化问题，后在国际上被称为中国邮路问题(Chinese postman problem 即CPP)。已知图 $G=(V, E)$ ，对于每条边 $e \in E$ ，有距离 $d(e)$ ，从 G 的某节点出发，走过 G 的所有边(允许重复穿过)，回到原出发点，使其总行程最短^[1]

2. 基本介绍

一个邮递员负责多条街，每条街上有很多投递点，以街道边，两条街交叉点为结点，该问题为欧拉问题，如果每个点度数为偶数则存在欧拉回路，从邮局出发，每条边即每条街道只走一次便回到邮局。如果有些结点度数为奇数，则要走回头路，在这种情况下如何找到一条最短路线。

二. 预备知识

1. 欧拉图

图 G 中经过每条边一次并且仅一次的回路称作欧拉回路。存在欧拉回路的图称为欧拉图。

2. 无向图欧拉图判定

无向图 G 为欧拉图，当且仅当 G 为连通图且所有顶点的度为偶数。

3. 有向图欧拉图判定

有向图 G 为欧拉图，当且仅当 G 的基图连通，且所有顶点的入度等于出度。

4. 欧拉回路性质：

性质1 设 C 是欧拉图 G 中的一个简单回路，将 C 中的边从图 G 中删去得到一个新的图 G' ，则 G' 的每一个极大连通子图都有一条欧拉回路。

性质2 设 C_1 、 C_2 是图 G 的两个没有公共边，但有至少一个公共顶点的简单回路，我们可以将它们合并成一个新的简单回路 C' 。 [2]

三. 算法思想

判断一条回路为最佳邮路的充要条件：

- ① C 中每条边最多重复一次。
- ② G 中任意回路上，重复边的长度不大于该回路一半。

基于该定理得到如下构造方法：

(1) 按每边最多重复一次的原则，使每点度数为偶数，这时存在欧拉回路，是可行解。

(2) 对于不满足条件②的回路取反，即原来重复的边不再重复，不重复的边要重复，同时要满足每点度数为偶数。 [3]

四. 算法实现

要解决上述问题，应分下面两大步骤。

1. 增加一些边，使得新图无奇度数结点，称这一步为可行方案
2. 调整可行方案，使其达到增加的边的总权值最小，称这个方案为最佳方案

代码算法：

1. 找出奇度顶点：统计每个节点的度数，并将度数为奇数的节点收集到 odd 数组中。
 2. 计算奇点间最短路径：对每个奇点运行 Dijkstra，得到它们之间的最短距离矩阵 cost。
 3. 最小权完美匹配：在奇点构成的完全图上，调用 Edmonds Blossom 算法求得一组最小权匹配对，使得每对奇点之间的路径代价和最小。
 4. 构造多重图：在原图基础上，复制匹配对之间的最短路径所对应的边，使得所有顶点度数均为偶数。
 5. 寻找欧拉回路：对这个增广后的多重无向图，使用 Hierholzer 算法（栈 + 回溯）找出一条经过所有边恰好一次的欧拉回路。
- 最终输出的 circuit 即为所求的欧拉回路序列。



四. 算法实现

代码:

```
struct Edge { int to; ll weight; };
// 原图邻接表
vector<vector<Edge>> graph;
// Dijkstra 单源最短路径
vector<ll> dijkstra(int src) {
    int n = graph.size();
    vector<ll> dist(n, INF);
    dist[src] = 0;
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto &e : graph[u]) {
            if (dist[e.to] > d + e.weight) {
                dist[e.to] = d + e.weight;
                pq.push({dist[e.to], e.to});
            }
        }
    }
    return dist;
}

// 最小权重完美匹配: 在完全图上对奇度顶点配对
// 需使用 Edmonds Blossom 算法实现, 或调用第三方库
vector<pair<int,int>> minWeightPerfectMatching(const vector<vector<ll>>& cost);
// 寻找欧拉回路 (多重无向图)
vector<int> findEulerianCircuit(multimap<int,int>& adj, int start) {
    vector<int> circuit, stack;
    stack.push_back(start);
    while (!stack.empty()) {
        int u = stack.back();
        auto it = adj.find(u);
        if (it != adj.end()) {
            int v = it->second;
            // 删除 u->v 和 v->u
            adj.erase(it);
            auto it2 = adj.find(v);
            for (; it2 != adj.end() && it2->second != u; ++it2);
            if (it2 != adj.end()) adj.erase(it2);
            stack.push_back(v);
        } else {
            circuit.push_back(u);
            stack.pop_back();
        }
    }
    reverse(circuit.begin(), circuit.end());
    return circuit;
}
```



四. 算法实现

代码:

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int N, M;
    // 输入: 节点数 N, 边数 M
    cin >> N >> M;
    graph.assign(N, {});
    vector<tuple<int,int,ll>> edges;
    for (int i = 0; i < M; i++) {
        int u, v;
        ll w;
        cin >> u >> v >> w;
        --u; --v;
        graph[u].push_back({v, w});
        graph[v].push_back({u, w});
        edges.emplace_back(u, v, w);
    }

    // 1. 找奇度顶点
    vector<int> degree(N, 0), odd;
    for (auto &t : edges) {
        int u,v; ll w;
        tie(u,v,w) = t;
        degree[u]++;
        degree[v]++;
    }
    for (int i = 0; i < N; i++) if (degree[i] % 2 == 1) odd.push_back(i);

    // 2. 计算奇点间最短路径矩阵
    int m = odd.size();
    vector<vector<ll>> cost(m, vector<ll>(m, 0));
    vector<vector<ll>> dists;
    for (int i = 0; i < m; i++) dists.push_back(dijkstra(odd[i]));
    for (int i = 0; i < m; i++) {
        for (int j = i+1; j < m; j++) {
            cost[i][j] = cost[j][i] = dists[i][odd[j]];
        }
    }
}
```




四. 算法实现

代码:

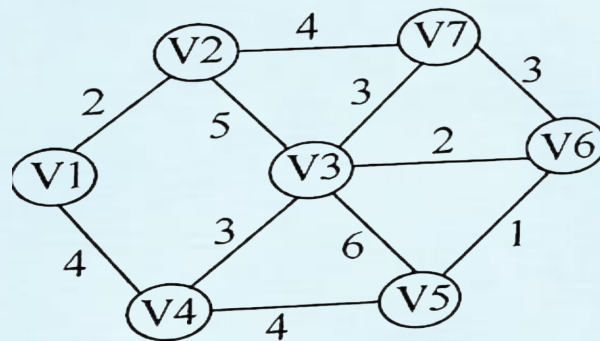
```
// 3. 在奇点完全图上做最小权匹配
auto match = minWeightPerfectMatching(cost);

// 4. 构造多重图 (原图 + 匹配路径复制)
multimap<int,int> adj;
// 插入原边
for (auto &t : edges) {
    int u,v; ll w;
    tie(u,v,w) = t;
    adj.insert({u, v});
    adj.insert({v, u});
}
// 5. 找欧拉回路
int start = odd.empty() ? 0 : odd[0];
auto circuit = findEulerianCircuit(adj, start);
// 输出
cout << "Eulerian circuit: ";
for (int u : circuit) cout << u+1 << " ";
cout << '\n';
return 0;
```

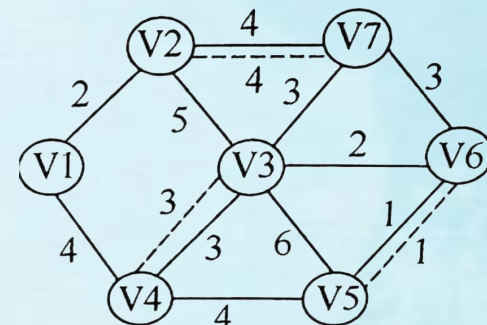
```
}
```

五. 求解过程

例.如图1.1所示, 求中国邮路的解



1.1



1.2

第一步

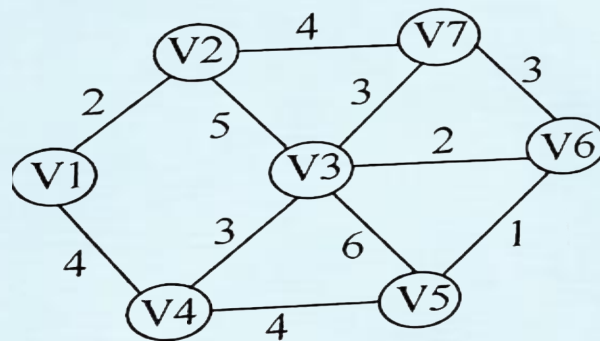
增加一些边, 使得新图无奇度数结点, 得到可行方案

图1.1中只有V1度数为偶数, 所以不存在欧拉回路。

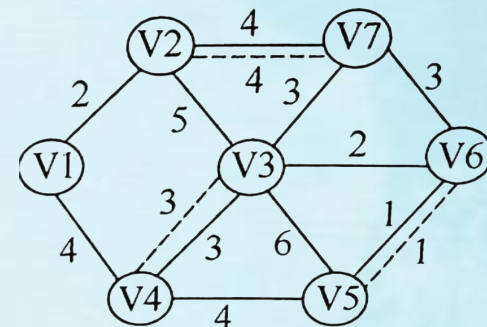
(1) 将与奇数结点相邻接的最短边 $\langle V5, V6 \rangle$ 重复一次, 这时V5、V6度数为偶数。将 $\langle V3, V4 \rangle$ 重复一次, V3、V4度数为偶数。若重复 $\langle V3, V6 \rangle$ 则V6变成奇数, 故不选。按此原则再重复 $\langle V2, V7 \rangle$, 这时每点度数为偶数, 存在欧拉回路, 如图1.2所示。

五. 求解过程

例.如图1.1所示, 求中国邮路的解



1.1



1.2

第二步

调整可行方案, 使其达到增加的边的总权值最小, 得到最佳方案

(2)判断每个回路是否为最短, 对于一般图可根据关联矩阵与找出所有回路, 对于点数较少的图用肉眼观察。

V1—V2—V3—V4—V1 中: 重复边 $3 \leq (2+4+5+3) / 2$

V2—V7—V3—V2 中: 重复边 $4 \leq (5+3+4) / 2$

V3-V6—V5—V3 中: 重复边 $1 \leq (6+1+2) / 2$

V3—V4—V5—V3 中: 重复边 $3 < (3+6+4) / 2$

V3-V6—V5—V4—V3 中: 重复边 $(3+1) < (3+1+2+4) / 2$

所有回路中重复边的长度 $\leq$ 该回路一半。故是最佳邮路。

六. 性能分析

1. Dijkstra 多次最短路

```
for (int i = 0; i < m; i++) dists.push_back(dijkstra(odd[i]));
```

单次 Dijkstra (使用二叉堆) : $O(m \log n)$

时间: 需要对 k 个奇度顶点各跑一次, 合计
 $O(k(m \log n))$.

空间: 当前存储了 k 条长度为 n 的距离向量, $O(kn)$ 。

2. 构造奇点完全图的权值矩阵

```
for (i < k) for (j = i + 1 < k) cost[i][j] = ...;
```

时间: 双重循环 $O(k^2)$ 。

空间: 存储权值矩阵 `cost` 大小 $k \times k$ $O(k^2)$ 。

3. 最小权重完美匹配

```
auto match = minWeightPerfectMatching(cost);
```

用常见的 Edmonds Blossom 算法, 其时间复杂度约 $O(k^3)$ 。

空间额外开销可视为 $O(k^2)$

六. 性能分析

4. 构造多重图 (`multimap<int,int> adj`)

for (每条原边) `adj.insert(...)`

for (匹配路径) `adj.insert(...)`

时间:

原图边插入: $2m$ 次 `multimap::insert`, 每次 $O(\log(m+kn))$ 合计约 $O(m\log m)$ 。

匹配路径插入的边数最多 $O(kn)$, 复杂度同理 $O(kn\log(m+kn))$ 。

空间: 存储所有多重边条目, $O(m+kn)$

5. 欧拉回路

```
while (!stack.empty()) {  
    auto it = adj.find(u);  
    adj.erase(it);  
}
```

每条有向条目被 `find` 和 `erase` 各一次

时间: $O((m+kn)\log(m+kn))$ 。

空间: 使用一个栈和结果向量, 共 $O(n+m+kn)$ 。

六. 性能分析

结论:

时间复杂度

$O(km\log n + k^3 + (m+kn)\log(m+kn))$, 稀疏图常见为 $O(n^3)$

空间复杂度

$O(m+kn+k^2)$, 最坏为 $O(n^2+m)$ 。

优点

能得到全局最优解: 通过最短路径配对奇点, 保证增广后欧拉回路总代价最小。

通用性强: 适用于任意带权无向图, 能够处理稠密或稀疏图。

步骤清晰: 分离奇点匹配和欧拉回路查找, 逻辑结构明晰, 易于模块化实现。

缺点

计算开销大: 需要对每个奇点跑 Dijkstra 并执行 Blossom 算法, 最坏情况下时间复杂度可达 $O(N^3\log N)$ 。内存消耗高: 存储多次 Dijkstra 的距离矩阵和匹配代价矩阵, 以及增广后的多重边, 一些大图上可能难以承受。

实现复杂: Edmonds Blossom 算法和路径重构细节较多, 对编码和调试要求较高。

七. 实际应用

- 城市环卫与道路维护

扫街道路清扫：环卫车辆需要将城市所有道路至少扫一次，CPP 能为扫街路线规划提供最省时省油的回路。

除雪作业：冬季除雪车要对所有道路进行清理，CPP 帮助确定除雪顺序、减少重复往返。

- 邮政与快递分发

邮件投递：邮递员要在社区或城区将信件、包裹投递到每条街道上的投递箱，CPP 确定最短回路以减少行走或驾驶里程。

快递分拣配送：在大规模园区内，比如校区、物流园内的末端配送，也可用 CPP 规划送件路线。

这些应用的共同特点是“需覆盖所有边（道路、管道、通道等）至少一次”且要“行程最短、成本最低”。CPP 及其变体（有向、半欧式、多车辆、多目标）在交通规划、机器人导航、物流调度、公共服务等领域都有深入研究和实际部署。

参考文献

- [1]. 数学辞海编辑委员会. 数学辞海第二卷. 中国科学技术出版社. 2002
- [2]. <https://blog.csdn.net/huangwei1024/article/details/1776866>
- [3]. 管梅谷. 奇偶点图上作业法. 数学学报, 1960, 10(3): 263-266
<https://doi.org/10.12386/A1960sxxb0020>

Thanks!

